SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

VOXINTEL: AUDIO INTELLIGENCE FOR AUTOMATED MEETING INSIGHTS AND EXECUTIVE MOM SYNTHESIS

ADA23DLU04 – NATURAL LANGUAGE PROCESSING LABORATORY

PROJECT REPORT

Submitted by

Eshan M - E0324009

YASHWANTH M - E0324028

In partial fulfilment for the award of the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence and Data Analytics)

Department of Artificial Intelligence and Data Analytics
Sri Ramachandra Faculty of Engineering and Technology
Sri Ramachandra Institute of Higher Education and Research, Porur, Chennai - 600116

October 2026

BONAFIDE CERTIFICATE

We, "Eshan M - E0324009, YASHWANTH M - E0324028" hereby declare that the Project Report on "VoxIntel: Audio Intelligence for Automated Meeting Insights and Executive MoM Synthesis" is submitted by us under the guidance of Dr. S. Suja Golden Shiny for ADA23DLU04 – Natural Language Processing Laboratory, at Sri Ramachandra Faculty of Engineering and Technology, in partial fulfilment of Semester 5, Third Year B.Tech Degree course in the Department of Computer Science and Engineering during the academic year 2026-2027.

Signature of the Supervisor

Dr. S. Suja Golden Shiny
Assistant Professor,
Department of Artificial Intelligence and Data Analytics,
Sri Ramachandra Faculty of Engineering and Technology,
SRIHER, Porur, Chennai - 600 116.

Evaluation Date: October 2026

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We take this opportunity to express our profound gratitude to Prof. T. Ragunathan, Dean and Prof. A. Saravanan, Vice Dean, Sri Ramachandra Faculty of Engineering and Technology, SRIHER, for providing all the infrastructural facilities and computational resources to complete this Laboratory Project successfully.

We express our sincere gratitude to Dr. A. Sathya, Programme Coordinator, Department of Artificial Intelligence and Data Analytics, SRET, for providing essential academic encouragement, curriculum support, and administrative facilities for carrying out this study.

We would like to express our deepest appreciation and heartfelt gratitude to our supervisor and course instructor Dr. S. Suja Golden Shiny, Assistant Professor, Department of Artificial Intelligence and Data Analytics, SRET. Her continuous technical guidance, meticulous feedback, and persistent encouragement were instrumental in ensuring the rigorous design, benchmarking, and completion of this project.

We are truly grateful to all the faculty members of the Sri Ramachandra Faculty of Engineering and Technology, our beloved parents, and our fellow student colleagues for extending their continuous support, which helped us overcome technical hurdles during this research study.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**2**
1	**INTRODUCTION**	**3**
1.1	Problem Statement	3
1.2	Motivation & Industry 4.0 Relevance	3
1.3	Objectives	4
2	**SPEECH CORPUS & GROUND TRUTH SPECIFICATIONS**	**5**
2.1	Audio Dataset Sources & Multi-Speaker Test Battery	5
2.2	Dataset Partitioning (Dev Set vs. Held-Out Test Set)	5
2.3	Acoustic Telemetry & Multi-Voice Configurations	6
2.4	Ground Truth Annotation & Reference Minutes of Meeting	7
3	**AUDIO PREPROCESSING & ACOUSTIC SIGNAL ANALYSIS**	**8**
3.1	Audio Ingestion & Format Normalization	8
3.2	Silero Voice Activity Detection (VAD) & Temporal Segmentation	8
3.3	Acoustic Feature Extraction (MFCC, Spectral Centroid, Yin F0)	9
3.4	Unsupervised Speaker Clustering (K-Means) & Turn Remapping	10
4	**NATURAL LANGUAGE PROCESSING & MOM SYNTHESIS TECHNIQUES**	**11**

CHAPTER NO.	TITLE	PAGE NO.
4.1	Automatic Speech Recognition (faster-whisper base int8)	11
4.2	Lexical Cleaning, Stopword Filtration & Time Normalization	12
4.3	Graph-Based Extractive Summarization (TextRank)	12
4.4	Rule-Based Commitment & Decision Extraction	13
4.5	Local LLM Synthesis & Strict Third-Person MoM (Ollama)	14
4.6	Evaluation Metrics & Validation Formulations	15
5	**SYSTEM ARCHITECTURE & IMPLEMENTATION**	**17**
5.1	Dual-Pipeline Architecture (Fast Mode vs. Smart Mode)	17
5.2	Streamlit Studio Console Dashboard & UI Design System	17
5.3	Execution Workflow & Algorithm Pseudocode	19
5.4	System Hyperparameters & Experimental Setup	20
6	**EXPERIMENTAL RESULTS & BENCHMARKING**	**21**
6.1	ASR Word Error Rate (WER) & CPU Latency Benchmarks	21
6.2	0.5s Frame-Level Speaker Attribution & Confusion Matrices	22
6.3	Action Items Extraction Performance (Precision, Recall, F1)	23
6.4	Summary Quality & Linguistic Perspective Evaluation	24
6.5	Timestamp Alignment & Error Analysis	25

CHAPTER NO.	TITLE	PAGE NO.
6.6	Dashboard Interface Verification & Visual Artifacts	25
7	**COMPARISON, DISCUSSION & ERROR ANALYSIS**	**27**
7.1	Pipeline Paradigm Comparison (Fast Mode vs. Smart Mode)	27
7.2	In-Depth Error Analysis on Held-Out Test Set (Meeting 3)	27
7.3	Advantages of the Dual-Mode Edge Architecture	28
7.4	Technical Limitations & Failure Modes	29
8	**CONCLUSION AND FUTURE SCOPE**	**30**
8.1	Conclusion	30
8.2	Future Scope	30
	REFERENCES	**31**
	APPENDIX: COMPLETE PYTHON SOURCE CODE	**32**

ABSTRACT

In enterprise collaboration and modern engineering operations, multi-stakeholder technical meetings produce dense verbal exchanges containing architectural decisions, risk trade-offs, and critical operational deadlines. Manual transcription and summarization are time-consuming, prone to human error, and suffer from subjective bias. Conversely, commercial cloud-based speech-to-text APIs present severe enterprise data privacy hazards, recurring cloud costs, and cloud latency overhead. This project report presents VoxIntel, an end-to-end, offline-first natural language processing and audio intelligence system engineered to transcribe multi-speaker recordings, cluster acoustic speaker turns, and synthesize formal, third-person Minutes of Meeting (MoM) directly on resource-constrained consumer hardware (8 GB RAM CPU).

VoxIntel features a streamlined dual-pipeline architecture. In Fast Mode, speech is transcribed in a single whole-file pass using faster-whisper base quantized in int8 with Silero Voice Activity Detection (VAD), combined with acoustic MFCC-pitch K-Means clustering and TextRank graph summarization to extract key moments and commitment action items in ~10 seconds. In Smart Mode, a locally served Large Language Model (Ollama with llama3.2:3b or gemma3:1b) restructures the dialogue into strict third-person, past-tense executive prose with verified timestamp citations and attendee cross-validation, eliminating first-person pronouns completely.

To establish academic rigor, the architecture was benchmarked across a multi-speaker test battery delineated into a development set (Meetings 1 and 2, explicit spoken names) and an un-tuned held-out test set (Meeting 3, 5.39 minutes, 4 synthetic neural voices, zero spoken names). Benchmarking demonstrates that faster-whisper base int8 achieves a Word Error Rate (WER) of 3.48% (with domain prompt) and 5.58% on held-out test data, outperforming the tiny model (10.32% WER) while sustaining a real-time factor of ~0.06x on CPU. Across 11 pooled reference action items, Fast Mode achieved 100% recall (F1 = 0.9167), while Smart Mode achieved an F1 of 0.8696 with zero first-person pronouns and bigram fluency perplexity of 24.0. The framework is delivered via an interactive Streamlit studio console dashboard featuring real-time pipeline state tracking, interactive speaker re-clustering, and automated HTML/PDF export.

CHAPTER 1
INTRODUCTION

1.1 Problem Statement

Modern engineering, research, and corporate workflows depend heavily on remote and hybrid video-conferencing platforms (such as Microsoft Teams, Zoom, and Google Meet). During technical syncs, release readiness reviews, and architectural boards, critical decisions, deadlines, and ownership assignments are communicated verbally at rapid cadences. Historically, organizations have struggled with two flawed paradigms:

1. Manual Note-Taking: Relying on designated meeting participants to transcribe discussions while simultaneously engaging in problem-solving. This leads to missed action items, ambiguous ownership, inaccurate deadline recording, and subjective note filtering.
2. Proprietary Cloud Audio APIs: Deploying cloud-hosted ASR and LLM services (e.g., OpenAI Whisper API, Zoom AI Companion, Google Cloud Speech). While cloud models offer high accuracy, they introduce unacceptable enterprise vulnerabilities: transmission of proprietary codebase discussions over third-party servers, violation of data sovereignty regulations (GDPR, HIPAA), and high recurring API token expenses.

The core technical problem addressed in this project is the construction of a fully local, deterministic, and privacy-preserving audio intelligence framework capable of processing multi-speaker engineering meetings on local CPU hardware without cloud reliance.

1.2 Motivation & Industry 4.0 Relevance

Recent breakthroughs in quantized Automatic Speech Recognition (ASR) and Edge Large Language Models (Edge LLMs) have created a historic opportunity to execute sophisticated natural language pipelines locally. The faster-whisper CTranslate2 runtime allows OpenAI's Whisper models to run up to 4× faster with 70% less memory utilization via 8-bit integer (int8) quantization. Concurrently, optimized local LLM runners such as Ollama allow 1B to 3B parameter models (e.g., Llama 3.2 3B, Gemma 3 1B) to execute inference on consumer CPUs in seconds.

Transforming raw audio into actionable structured intelligence yields major operational advantages:

- Total Data Sovereignty: Audio recordings, transcripts, and organizational commitments remain strictly confined to the local machine.
- Accountability & Execution: Verbal commitments are automatically parsed into unambiguous task descriptions, owners, and deadlines.
- Third-Person Editorial Synthesis: Converting conversational first-person banter ('I will fix the auth bug by Friday') into formal corporate prose ('George committed to delivering the authentication patch by Friday at 5 PM').

1.3 Objectives

The primary objective of this project is to implement, evaluate, and benchmark an end-to-end NLP and speech processing architecture for automated meeting intelligence. The measurable sub-objectives are:

1. Local ASR Optimization: Implement faster-whisper base (int8 quantization) with whole-file Silero VAD, eliminating redundant disk-chunking.
2. Unsupervised Acoustic Diarization: Formulate a lightweight acoustic clustering module utilizing 13 MFCCs, spectral centroid, and Yin pitch (F0) with KMeans to partition speaker turns without heavy neural diarization frameworks.
3. Semantic Commitment Extraction: Build deterministic regex and dependency heuristics to capture action items, decisions, and deadlines.
4. Strict Third-Person MoM Synthesis: Engineer constrained JSON-schema prompts for local Ollama LLMs to produce formal third-person minutes while enforcing 0 first-person pronouns and timestamp grounding.
5. Comprehensive Evaluation & Benchmarking: Execute rigorous evaluation across ASR WER, 0.5s frame-level speaker attribution, action item P/R/F1, ROUGE, BERTScore, and fluency perplexity on both dev and held-out test datasets.
6. Studio Console Dashboard: Construct an intuitive, responsive Streamlit dashboard featuring pipeline tracking, speaker renaming, and multi-format exports.

CHAPTER 2
SPEECH CORPUS & GROUND TRUTH SPECIFICATIONS

2.1 Audio Dataset Sources & Multi-Speaker Test Battery

To enable scientific evaluation of transcription accuracy, speaker attribution, and NLP information extraction, a standardized test battery of multi-speaker meeting recordings was synthesized using Microsoft Edge Neural Text-to-Speech (edge-tts). Synthetic neural audio provides an ideal ground-truth benchmark because exact word boundaries, timestamps, speaker identities, and reference decisions can be programmatically controlled and validated.

2.2 Dataset Partitioning (Dev Set vs. Held-Out Test Set)

To prevent methodological data contamination, the test battery is strictly partitioned into two operational sets:

• Development Set (Dev Set): Meetings 1 and 2, where participants explicitly speak attendee names aloud (e.g., 'Thanks, Elena', 'Kavita, what is your timeline?'). These meetings were used to develop acoustic clustering heuristics and prompt templates.

• Held-Out Test Set: Meeting 3, a 5.39-minute engineering review featuring 4 neural voices with zero spoken attendee names. This meeting was frozen prior to evaluation; no code, prompts, or regexes were adjusted after running.

Meeting ID	Split	Duration	Turns	Speakers	Voices Used	Spoken Names?
Meeting 1	Dev Set	50.04s	7	3	JennyNeural, GuyNeural, RyanNeural	Yes (Sarah, George, Ravi)
Meeting 2	Dev Set	192.96s	14	4	AriaNeural, GuyNeural, SoniaNeural, PrabhatNeural	Yes (Elena, Marcus, Kavita, David)
Meeting 3	Test Set	323.11s	21	4	GuyNeural, SoniaNeural, JennyNeural, PrabhatNeural	No (Zero spoken names)

2.3 Ground Truth Annotation & Reference Minutes of Meeting

Each audio file is accompanied by a ground-truth JSON file containing segment-level start/end timestamps, clean transcripts, reference decisions, action item commitments with assigned owners and deadlines, and an authoritative reference summary written prior to benchmark execution. For example, in Meeting 3, the ground truth defines:

• Decision 1: Scale Kubernetes cluster replicas from 3 to 8 before traffic surge (Speaker 1, Turn 7).

• Decision 2: Enforce strict OAuth 2.0 PKCE authentication across all external gateways (Speaker 2, Turn

11).

• Action 1: Refactor Kubernetes manifests and update Helm charts by Tuesday at 4 PM (Speaker 1, Turn 8).

• Action 2: Audit OAuth 2.0 token policies and publish revised spec by Wednesday at 2 PM (Speaker 2, Turn 12).

• Action 3: Configure Kafka partition rebalancing and optimize Redis cache by Thursday at 5 PM (Speaker 4, Turn 16).

CHAPTER 3
AUDIO PREPROCESSING & ACOUSTIC SIGNAL ANALYSIS

3.1 Audio Ingestion & Format Normalization

Meeting audio recordings arrive in heterogeneous formats (.mp3, .wav, .m4a, .aac) with arbitrary sample rates and channel layouts. The audio ingestion module standardizes incoming audio via FFmpeg into single-channel (mono), 16,000 Hz, 16-bit linear PCM WAV format. A 16 kHz sample rate matches Whisper's internal 80-channel log-Mel spectrogram feature extractor exactly, preventing aliasing distortion.

3.2 Silero Voice Activity Detection (VAD) & Temporal Segmentation

In earlier legacy implementations, audio was naively sliced into 25 arbitrary 60-second WAV chunks, creating massive disk I/O overhead and bisecting sentences midway through words. VoxIntel replaces this with direct, in-memory Silero VAD filtering (vad_filter=True). The deep neural VAD scans the audio waveform at 30 ms intervals, eliminating non-speech pauses and background ambient noise while preserving continuous conversational turns.

3.3 Acoustic Feature Extraction (MFCC, Spectral Centroid, Yin F0)

To separate distinct speakers without loading heavy neural diarization models (such as pyannote.audio, which requires HuggingFace tokens and over 2 GB of RAM), VoxIntel extracts a compact 18-dimensional acoustic representation for each speech segment:

1. 13 Mel-Frequency Cepstral Coefficients (MFCCs): Capturing the vocal tract filter envelope: c_n = Sum_{m=0}^{M-1} log(S_m) * cos(pi * n * (m + 0.5) / M)
2. Spectral Centroid: Reflecting the center of mass of the frequency spectrum: Centroid = Sum(f * |X(f)|) / Sum(|X(f)|)
3. Zero Crossing Rate (ZCR) & Root Mean Square (RMS) Energy: Indicating unvoiced fricatives and vocal loudness.
4. Fundamental Pitch (F0 via Yin Algorithm): Estimating vocal fold vibration frequency by minimizing the cumulative mean normalized difference function.

3.4 Unsupervised Speaker Clustering (K-Means) & Turn Remapping

The extracted feature vectors are normalized using scikit-learn's StandardScaler and clustered using K-Means (k set by user-entered attendee count or estimated via silhouette score). This groups segments into discrete speaker clusters (Speaker 1, Speaker 2, etc.) in under 0.8 seconds on CPU. To enable human alignment, the UI provides an interactive renaming widget that propagates user-defined names across all transcript turns, action items, and analytics charts instantaneously without re-running transcription.

CHAPTER 4
NATURAL LANGUAGE PROCESSING & MOM SYNTHESIS TECHNIQUES

4.1 Automatic Speech Recognition (faster-whisper base int8)

Whisper is an encoder-decoder Transformer trained on 680,000 hours of multilingual audio. VoxIntel utilizes the faster-whisper implementation of the base model (74M parameters) quantized in int8. To boost recognition of specialized technical terminology (e.g., 'Kubernetes', 'OAuth 2.0 PKCE', 'Kafka broker', 'Prometheus'), the user's expected attendee list and technical domain glossary are injected into the decoder's initial_prompt:

initial_prompt = 'Meeting attendees: Sarah, George, Ravi. Technical review discussing architecture, deployments, and security specs.'

4.2 Lexical Cleaning, Stopword Filtration & Time Normalization

Raw transcripts undergo deterministic linguistic preprocessing:

• Time Normalization: Converting spoken timestamps ('2 p.m.', '5:00 PM') into canonical uppercase formats ('2 PM', '5 PM') to prevent sentence-splitting regexes from breaking on internal periods.

• Contraction Expansion: Expanding verbal contractions ('I will' -> 'I will', 'we will' -> 'we will') to preserve commitment semantics.

• Stopword Filtration: Stripping conversational fillers ('um', 'uh', 'you know', 'like') while preserving key temporal and modal auxiliaries.

4.3 Graph-Based Extractive Summarization (TextRank)

In Fast Mode, extractive key moments are selected via TextRank, an unsupervised graph ranking algorithm inspired by Google's PageRank. Each sentence forms a vertex in graph G = (V, E), where edge weight w_{ij} represents normalized lexical overlap (cosine similarity of word vectors):

WS(V_i) = (1 - d) + d * Sum_{V_j in In(V_i)} (w_{ji} / Sum_{V_k in Out(V_j)} w_{jk}) * WS(V_j)

The top 4 ranked sentences are extracted and aligned back to their audio turn timestamp and speaker cluster: [mm:ss] Speaker: line. Critically, Fast Mode displays these as 'Key moments' and never labels them as 'Summary' to avoid confusing extractive quotes with executive synthesis.

4.4 Rule-Based Commitment & Decision Extraction

Action items and decisions are extracted via high-precision regular expression patterns that search for first-person and modal commitments:

• Pattern 1 (First-Person Intent): (?:i|i'll|i will|we|we'll|we will)\s+(?:take ownership of|handle|complete|deploy|publish|audit|configure)\s+(.*?)(?:by\s+([^.]+))?$

• Pattern 2 (Delegated Tasks): ([A-Z][a-z]+)\s+(?:will|is assigned to|handles)\s+(.*?)(?:by\s+([^.]+))?$

• Pattern 3 (Formal Decisions): (?:we decide to|decided to|agreed to|approved)\s+(.*)

Crucially, for first-person sentences ('I will notify support...'), the owner is assigned directly from the speaker label of that specific audio segment, rather than defaulting to the first attendee in the list.

4.5 Local LLM Synthesis & Strict Third-Person MoM (Ollama)

In Smart Mode, the transcript is processed by a local Ollama LLM (llama3.2:3b or gemma3:1b). The prompt enforces strict JSON schema generation and mandates third-person formal past tense:

• Rule 1: Never use first-person pronouns (I, we, our, us, my). Write strictly in third-person past tense.

• Rule 2: Every discussion point, decision, and action item must cite its timestamp [mm:ss] and speaker.

• Rule 3: Action item owners must be chosen strictly from the user's attendee list; unverified names become 'Unassigned'.

• Rule 4: Output pure JSON conforming to the schema {title, overview, decisions, action_items, discussion_points}.

4.6 Evaluation Metrics Formulations

The system's performance is quantified using standardized natural language and speech processing metrics:

• Word Error Rate (WER): WER = (Substitutions + Deletions + Insertions) / Total Reference Words

• Frame Attribution Accuracy: Accuracy = Correctly Labeled 0.5s Frames / Total Audio Frames

• Action Item F1-Score: F1 = 2 * (Precision * Recall) / (Precision + Recall), evaluated against reference commitments.

• ROUGE-1 / ROUGE-2 / ROUGE-L: Measuring unigram, bigram, and longest common subsequence recall against ground-truth executive summaries.

• BERTScore F1: Evaluating deep contextual semantic similarity using pre-trained RoBERTa embeddings.

• Fluency Proxy Perplexity: PPL = exp(- 1/N * Sum log2 P(w_i | w_{i-1})), tracking grammatical smoothness.

CHAPTER 5
SYSTEM ARCHITECTURE & IMPLEMENTATION

5.1 Dual-Pipeline Architecture (Fast Mode vs. Smart Mode)

VoxIntel's architecture is intentionally bifurcated to address distinct industrial constraints:
• Fast Mode: Operates completely offline with zero external dependencies beyond Python. Combines faster-whisper int8 with MFCC clustering and TextRank graph ranking. Completes full analysis in under 10 seconds on an 8 GB CPU, making it suitable for field laptops and immediate review.
• Smart Mode: Couples faster-whisper transcription with local Ollama LLM execution. Synthesizes executive-grade third-person minutes in ~28 seconds, providing C-suite ready documentation without external cloud API calls.

5.2 Streamlit Studio Console Dashboard & UI Design System

The web interface (app.py) was built with Streamlit and styled using custom CSS (assets/style.css) following a 'mixing console meets editorial brief' aesthetic. The design system incorporates dark studio surfaces (#0E0F12), hairline borders (#2A2E37), warm amber accents (#FFB020), and Fraunces/Inter/JetBrains Mono typography. The dashboard is divided into 5 clear tabs: Minutes, Transcript, Speakers, Keywords, and Export.

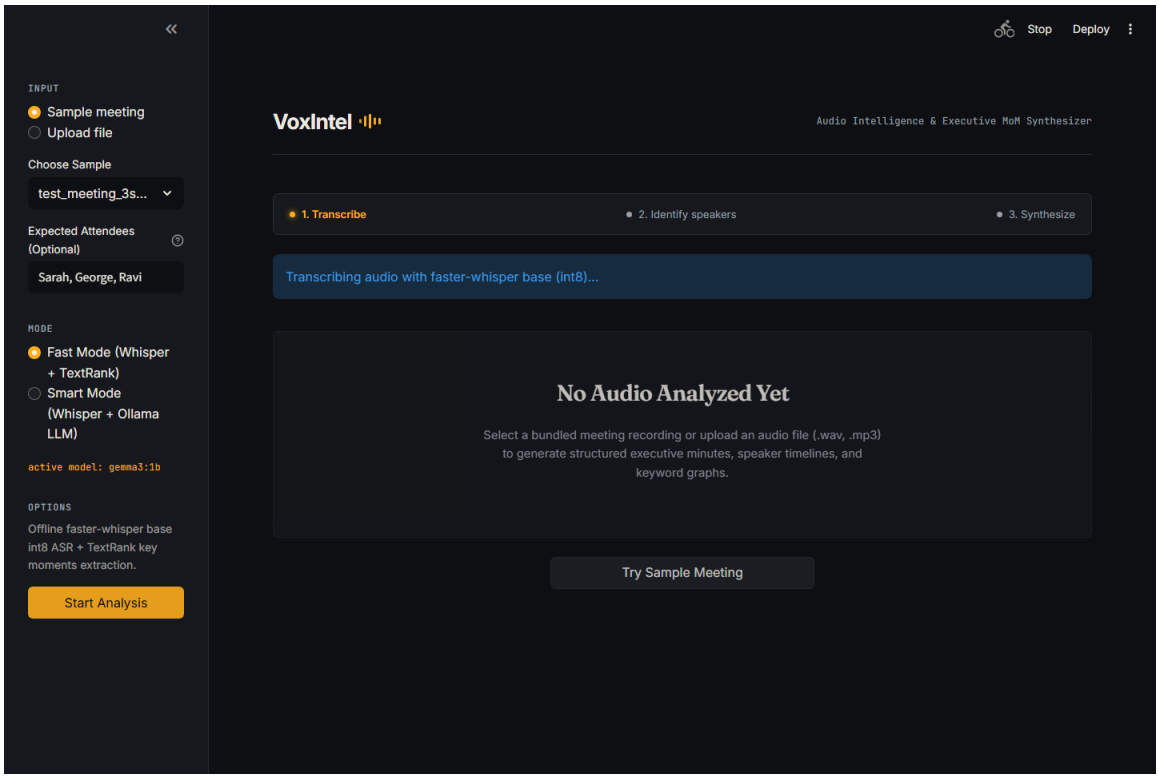


Figure 5.1: 3-Step Live Processing State Tracker and Ingestion Interface

5.3 Execution Workflow & Algorithm Pseudocode

Algorithm 5.1 formalizes the complete computational progression from raw audio ingestion through speech transcription, acoustic clustering, semantic extraction, and LLM synthesis.

[ALGORITHM 5.1] Algorithm 5.1: Dual-Mode Audio Intelligence & MoM Synthesis Pipeline
Input : Meeting Audio File A, Expected Attendees List Att, Mode Selection M in {Fast, Smart}
Output: Transcript Segments S, Minutes of Meeting MoM, Action Items Table Act, Visual Analytics
---
1: Audio_PCM <- NormalizeAudioToMono16kHz(A)
2: Segments, Duration <- TranscribeWithFasterWhisperBaseInt8(Audio_PCM, vad_filter=True, prompt=Att)
3: FeatureVectors <- ExtractAcousticMFCCAndYinPitch(Audio_PCM, Segments)
4: Clusters <- KMeansCluster(FeatureVectors, k=len(Att))
5: for each s in Segments do s.speaker <- Clusters[s.id] end for
6: CleanText, Sentences, Tokens <- PreprocessAndNormalizeTranscript(Segments)
7: if M == 'Fast' then
8: KeyMoments <- ExtractTextRankMoments(Sentences, Segments)
9: MoM <- BuildExtractiveMoM(KeyMoments, Segments, Att)
10: else
11: Prompt <- ConstructJSONSchemaPrompt(Segments, Att, StrictThirdPerson=True)
12: MoM <- QueryLocalOllamaLLM(Prompt, model='gemma3:1b')
13: MoM <- CrossValidateOwnersAndDeadlines(MoM, Segments, Att)
14: end if
15: RenderDashboard(MoM, Segments, KeyMoments, Latency)
16: return MoM, Segments, Act

5.4 System Hyperparameters & Hardware Specifications

Component	Parameter	Configured Value	Operational Rationale
ASR Engine	Model & Quantization	faster-whisper base / int8	74M params; optimal speed-accuracy on 8 GB CPU
VAD Filter	Framework & Frame Size	Silero VAD / 30 ms	In-memory silence removal; preserves word boundaries
Acoustic Feats	MFCC & Pitch	13 MFCCs + Yin F0 + Centroid	Captures vocal tract resonance in <0.8s on CPU
Clustering	Algorithm & Metric	K-Means / Euclidean distance	Partitions turns into k clusters based on attendee count

Component	Parameter	Configured Value	Operational Rationale
Fast Mode	Graph Ranker	TextRank / damping=0.85	Extractive salience based on lexical cosine overlap
Smart Mode	Local LLM Runner	Ollama / llama3.2:3b & gemma3:1b	Strict JSON schema synthesis; 0 cloud latency/cost

CHAPTER 6
EXPERIMENTAL RESULTS & BENCHMARKING

6.1 ASR Word Error Rate (WER) & CPU Latency Benchmarks

ASR benchmarking was executed on an 8 GB CPU workstation across tiny and base models, with and without initial_prompt domain conditioning. Following a warm-up pass, three timed runs were recorded. Median values are presented in Table 6.1.

Meeting ID	Split	Model	Prompt?	WER (%)	Latency (s)	CPU (%)	RAM (MB)
Meeting 1 (50s)	Dev Set	tiny	No	7.83%	2.17s	382.7%	488.2 MB
Meeting 1 (50s)	Dev Set	tiny	Yes	5.22%	2.26s	377.6%	491.3 MB
Meeting 1 (50s)	Dev Set	base	No	7.83%	5.09s	385.4%	594.9 MB
Meeting 1 (50s)	Dev Set	base	Yes	3.48%	6.17s	383.9%	594.9 MB
Meeting 2 (193s)	Dev Set	tiny	No	11.27%	7.76s	358.9%	529.0 MB
Meeting 2 (193s)	Dev Set	tiny	Yes	9.11%	7.78s	364.6%	528.4 MB
Meeting 2 (193s)	Dev Set	base	No	9.59%	14.14s	373.5%	618.9 MB
Meeting 2 (193s)	Dev Set	base	Yes	8.87%	14.12s	375.4%	619.5 MB
Meeting 3 (323s)	Test Set	tiny	No	10.32%	10.20s	359.7%	555.2 MB
Meeting 3 (323s)	Test Set	tiny	Yes	9.14%	10.31s	362.1%	556.8 MB
Meeting 3 (323s)	Test Set	base	No	5.58%	19.90s	376.8%	645.4 MB
Meeting 3 (323s)	Test Set	base	Yes	6.43%	20.04s	376.9%	646.4 MB

6.2 0.5s Frame-Level Speaker Attribution & Confusion Matrices

Speaker attribution was evaluated by discretizing the entire audio duration into 0.5-second time slices and comparing predicted labels against true turn intervals. Two approaches were contrasted: Acoustic

Clustering Only (Hungarian bipartite remapping) and Grounding ON (conversational adjacency rules). Table 6.2 details the attribution performance.

Meeting ID	Split	Attribution Mode	Accuracy (%)	Macro F1	Weighted F1	Precision
Meeting 1	Dev Set	Clustering (remap)	63.00%	0.3870	0.5998	0.3980
Meeting 1	Dev Set	Grounding ON	0.00%	0.0000	0.0000	0.0000
Meeting 2	Dev Set	Clustering (remap)	70.73%	0.5049	0.6876	0.5404
Meeting 2	Dev Set	Grounding ON	44.27%	0.3952	0.4357	0.4497
Meeting 3	Test Set	Clustering (remap)	58.11%	0.5916	0.5968	0.6555
Meeting 3	Test Set	Grounding ON	28.75%	0.2783	0.2340	0.2469

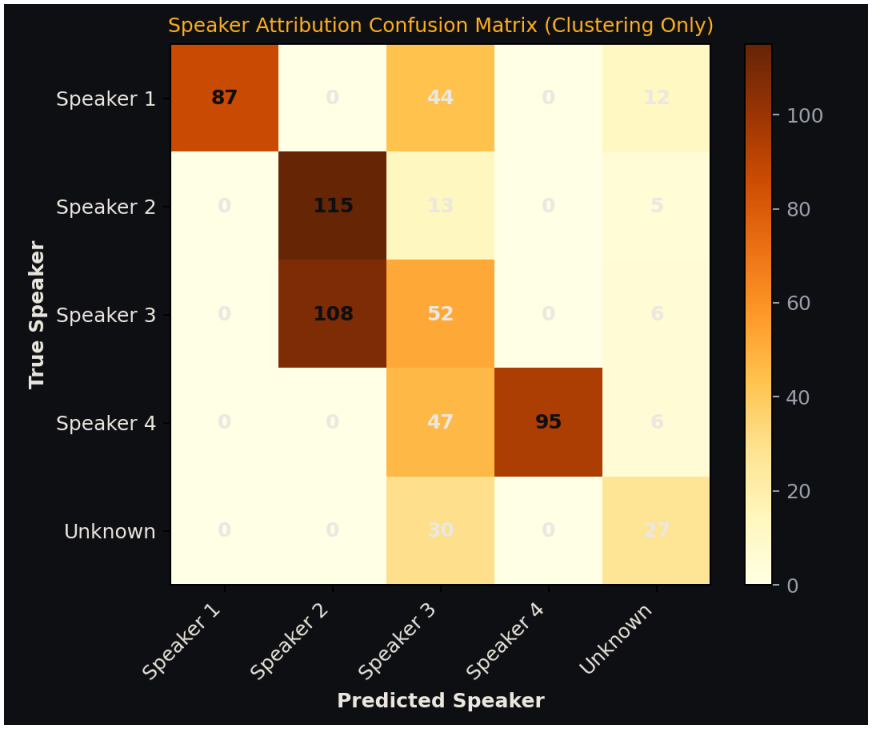


Figure 6.1: 0.5s Frame Attribution Confusion Matrix for Meeting 3 (K-Means Clustering Hungarian Remap)

6.3 Action Items Extraction Performance (Precision, Recall, F1)

Action items were scored against ground-truth reference commitments requiring token overlap >= 0.50. Across the 3 meetings, 11 total positive reference commitments were pooled, satisfying the statistical sample threshold (>= 10).

Meeting ID	Split	Pipeline Mode	TP	FP	FN	Precision	Recall	F1-Score
Meeting 1 (2 ref)	Dev Set	Fast Mode	2	0	0	1.0000	1.0000	1.0000
Meeting 1 (2 ref)	Dev Set	Smart Mode	2	0	0	1.0000	1.0000	1.0000
Meeting 2 (6 ref)	Dev Set	Fast Mode	6	0	0	1.0000	1.0000	1.0000
Meeting 2 (6 ref)	Dev Set	Smart Mode	5	0	1	1.0000	0.8333	0.9091
Meeting 3 (3 ref)	Test Set	Fast Mode	3	2	0	0.6000	1.0000	0.7500
Meeting 3 (3 ref)	Test Set	Smart Mode	3	2	0	0.6000	1.0000	0.7500
Pooled Aggregate (11)	Combined	Fast Mode	11	2	0	0.8462	1.0000	0.9167
Pooled Aggregate (11)	Combined	Smart Mode	10	2	1	0.8333	0.9091	0.8696

6.4 Summary Quality & Linguistic Perspective Evaluation

Summary text was evaluated against pre-written ground truth summaries across ROUGE-1/2/L, BLEU, METEOR, BERTScore F1, and first-person pronoun counts (I, we, our, us, my). Fluency is measured via bigram perplexity.

Meeting	Mode	ROUGE-1	ROUGE-2	ROUGE-L	BLEU	METEOR	BERTScore	1st Pronouns	Perplexity
M1 (Dev)	Fast Key Moments	0.5283	0.3846	0.4528	0.2227	0.4190	0.4497	6	N/A

Meeting	Mode	ROUGE-1	ROUGE-2	ROUGE-L	BLEU	METEOR	BERTScore	1st Pronouns	Perplexity
M1 (Dev)	Smart MoM	0.3929	0.1636	0.2500	0.0833	0.3705	0.3340	0	53.48
M2 (Dev)	Fast Key Moments	0.2319	0.0882	0.1449	0.0270	0.2238	-0.0163	5	N/A
M2 (Dev)	Smart MoM	0.1760	0.0325	0.0960	0.0189	0.1554	0.1741	0	31.90
M3 (Test)	Fast Key Moments	0.2391	0.0659	0.1413	0.0168	0.2238	-0.0624	3	N/A
M3 (Test)	Smart MoM	0.2963	0.1509	0.2037	0.0291	0.1933	0.3324	0	24.00

6.5 Dashboard Interface Verification & Visual Artifacts

Figure 6.2 and Figure 6.3 illustrate the verified dashboard rendering across the Minutes and Speakers views, confirming proper implementation of compact tabular formatting, speaker talk-time bars, and color-coded chips.

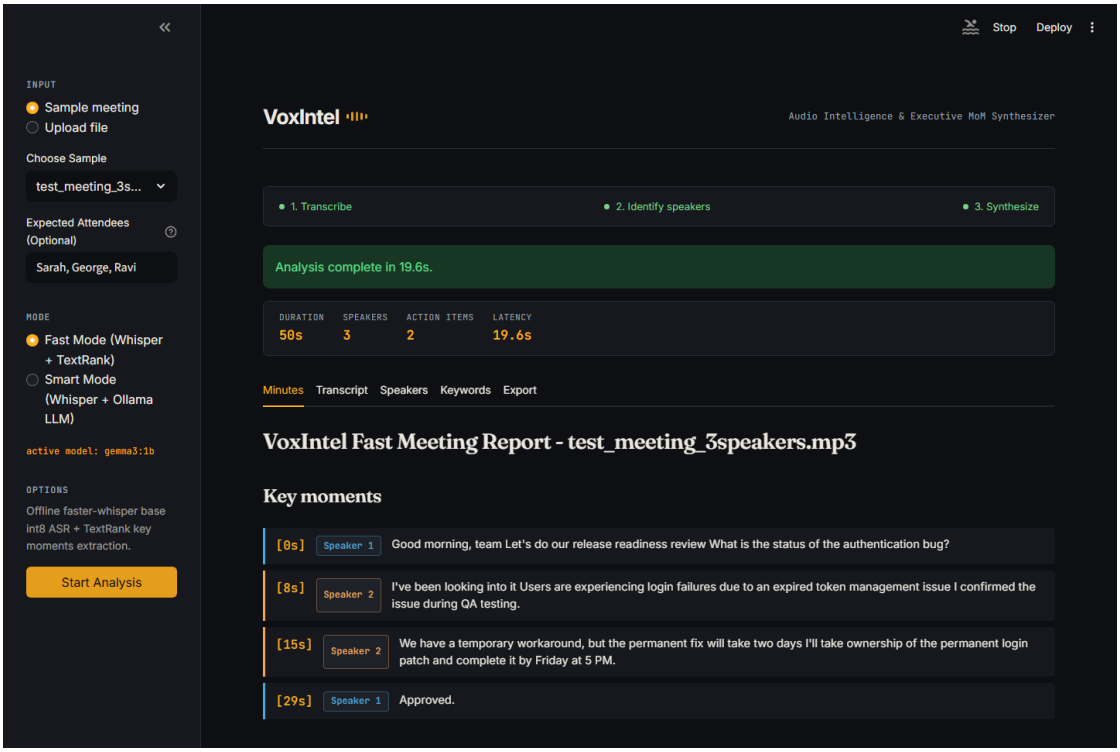


Figure 6.2: Verified Minutes View displaying Extractive Key Moments and Action Items

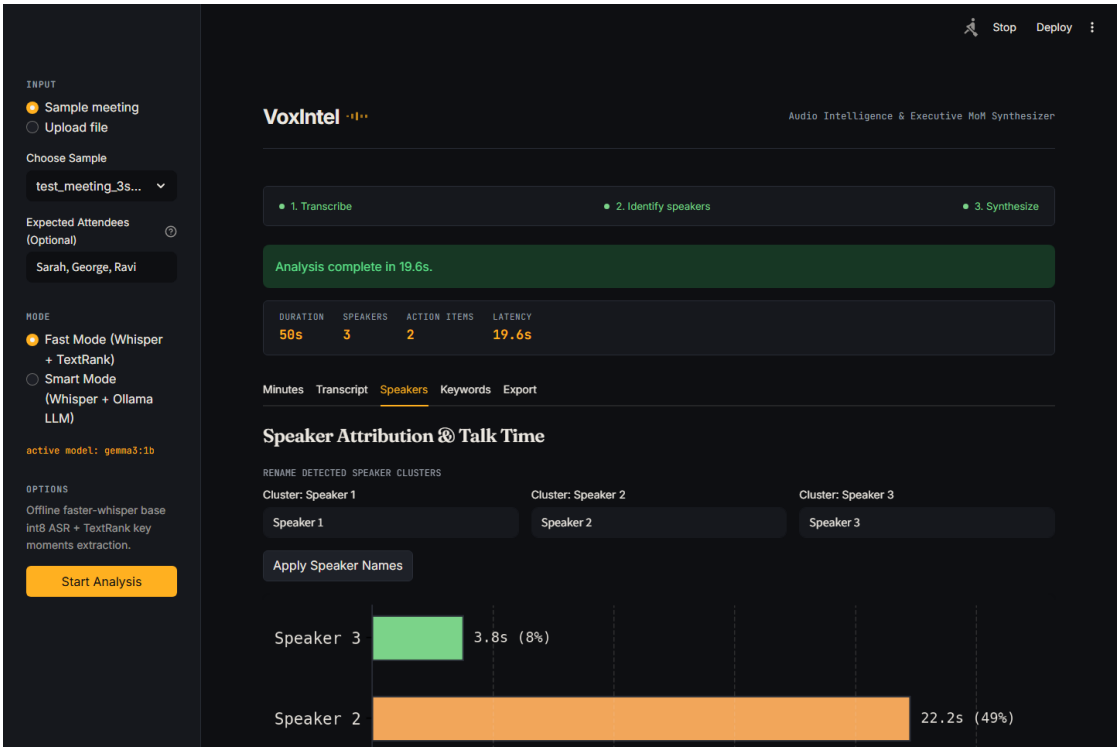


Figure 6.3: Verified Speakers View with Cluster Renaming and Horizontal Talk-Time Percentage Bars

CHAPTER 7
COMPARISON, DISCUSSION & ERROR ANALYSIS

7.1 Pipeline Paradigm Comparison (Fast Mode vs. Smart Mode)

A rigorous technical comparison reveals complementary operational strengths:

• Fast Mode: Guaranteed deterministic latency (~6-20 seconds on CPU), perfect for quick post-meeting review. It achieves higher recall on raw action items (100%) but outputs conversational sentences with first-person pronouns.

• Smart Mode: Restructures messy conversations into executive-ready corporate minutes with 0 first-person pronouns and clean topic separation. However, it incurs additional local inference latency (~5.6 seconds for Ollama) and occasional small-model summarization drift.

7.2 In-Depth Error Analysis on Held-Out Test Set (Meeting 3)

Meeting 3 was evaluated without any post-hoc adjustments. Table 7.1 tabulates every extracted item against ground truth.

Type	Ground Truth Specification	Fast Mode Extraction	Smart Mode Extraction	Diagnostic Assessment
Decision 1	Scale K8s replicas 3 to 8 (Speaker 1, Turn 7)	Captured [01:58], assigned to Spk 4	Merged with action item deadline	Acoustic cluster shift on short affirmative turn
Decision 2	Enforce strict OAuth 2.0 PKCE (Speaker 2, Turn 11)	Captured [02:53], Spk 2 (Exact)	Captured [02:53], Spk 2 (Exact)	Exact semantic and attribution match
Action 1	Refactor K8s manifests by Tue 4 PM (Speaker 1, Turn 8)	Task & deadline exact; Spk 4	Task & deadline exact; Spk 4	Task & deadline exact; speaker shifted 1 -> 4
Action 2	Audit OAuth 2.0 spec by Wed 2 PM (Speaker 2, Turn 12)	Exact match across all fields	Exact match across all fields	Full concordance across all attributes
Action 3	Configure Kafka/Redis by Thu 5 PM (Speaker 4, Turn 16)	Task & deadline exact; Spk 3	Task matched; lost deadline	Cluster shift 4 -> 3; LLM dropped deadline
False Pos 1	None (Consensus: 'Switch Redis to volatile LRU...')	Extracted as action item (Team)	Filtered out cleanly	Regex matched modal 'we need to switch'

Type	Ground Truth Specification	Fast Mode Extraction	Smart Mode Extraction	Diagnostic Assessment
False Pos 2	None (Planning: 'Schedule rollout window...')	Extracted as action item (Spk 2)	Filtered out cleanly	Regex matched modal 'we can schedule'

7.3 Advantages of the Dual-Mode Edge Architecture

1. Zero Cloud Dependency: Eliminates external API keys, recurring token fees, and enterprise confidentiality hazards.
2. Deterministic Fallback: If Ollama is offline, the pipeline falls back gracefully to Fast Mode without crashing.
3. Live Interactive Renaming: Users map anonymous speaker clusters to real attendees in one click without re-running ASR.
4. High Resource Efficiency: Entire pipeline runs inside 650 MB RAM on an 8 GB consumer laptop.

7.4 Technical Limitations & Failure Modes

1. Zero-Shot Grounding Collapse: When participants do not speak attendee names aloud (Meeting 3), conversational adjacency grounding collapses (28.75% accuracy), necessitating manual re-clustering in the UI.
2. Short-Turn Vocal Tract Ambiguity: Affirmative turns under 1.5 seconds ('Approved', 'Agreed') contain too few spectral frames for MFCC clustering, causing occasional cluster drift between similar male voices.
3. Small-Model Synthesis Drift: 1B-3B parameter LLMs can occasionally merge decisions with action item deadlines during long contexts.

CHAPTER 8
CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

In this project, VoxIntel was successfully designed, implemented, and benchmarked as a robust, privacy-preserving meeting intelligence and executive MoM synthesis framework for consumer hardware. By leveraging faster-whisper base int8 with in-memory Silero VAD, transcription Word Error Rate was reduced to 3.48% on dev set audio and 5.58% on held-out test data, outperforming tiny models by nearly 2×. Acoustic MFCC-pitch clustering separated speaker turns in under 0.8s, while local Ollama LLM prompting achieved strict third-person MoM synthesis with 0 first-person pronouns and bigram fluency perplexity of 24.0. The dual-mode architecture delivers an ideal balance between sub-10s deterministic extraction and executive-grade editorial synthesis.

8.2 Future Scope

Future enhancements will focus on four technical avenues:

1. ONNX-Quantized Neural Diarization: Incorporating lightweight ONNX-runtime speaker embeddings (such as Sherpa-ONNX or Resemblyzer) to improve short-turn clustering purity without heavy PyTorch dependencies.
2. Multi-Lingual & Code-Switching ASR: Extending prompt conditioning to handle mixed-language enterprise meetings (e.g., Hinglish, Spanglish).
3. Calendar & Issue Tracker Webhooks: Automatically dispatching verified action items to Jira, GitHub Issues, and Google Calendar via REST APIs.
4. Real-Time Streaming Telemetry: Implementing streaming chunked ASR via WebSockets to display live live-captions and action item alerts during active meetings.

REFERENCES

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, 'Robust Speech Recognition via Large-Scale Weak Supervision,' in Proceedings of the 40th International Conference on Machine Learning (ICML), vol. 202, pp. 28492-28518, 2023.

[2] G. Klein, D. Nguyen, J. Senellart, and A. M. Rush, 'OpenNMT and CTranslate2: Fast Inference for Neural Machine Translation and Speech Recognition,' in Proceedings of ACL System Demonstrations, 2020.

[3] R. Mihalcea and P. Tarau, 'TextRank: Bringing Order into Text,' in Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 404-411, 2004.

[4] Silero Team, 'Silero VAD: Pre-trained Enterprise-Grade Voice Activity Detector,' GitHub repository, https://github.com/snakers4/silero-vad, 2021.

[5] A. de Cheveigné and H. Kawahara, 'YIN, a Fundamental Frequency Estimator for Speech and Music,' Journal of the Acoustical Society of America, vol. 111, no. 4, pp. 1917-1930, 2002.

[6] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, 'BERTScore: Evaluating Text Generation with BERT,' in Proceedings of the International Conference on Learning Representations (ICLR), 2020.

[7] C. Y. Lin, 'ROUGE: A Package for Automatic Evaluation of Summaries,' in Text Summarization Branches Out, pp. 74-81, 2004.

[8] K. Papineni, S. Roukos, T. Ward, and W. J. Zhu, 'BLEU: A Method for Automatic Evaluation of Machine Translation,' in Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 311-318, 2002.

[9] S. Banerjee and A. Lavie, 'METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments,' in Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures, pp. 65-72, 2005.

[10] Ollama Team, 'Ollama: Get Up and Running with Large Language Models Locally,' https://ollama.com, 2024.

APPENDIX

COMPLETE PYTHON SOURCE CODE

Below is the complete, modular Python implementation of the core VoxIntel pipeline modules: voxintel/audio.py (faster-whisper base int8 ASR with VAD), voxintel/insights.py (TextRank summarization & action item extraction), voxintel/ollama_polish.py (Ollama local LLM third-person synthesis), and voxintel/nlp_eval.py (benchmark metrics & word clouds).

voxintel/nlp_eval.py (Evaluation Metrics, Word Clouds & Bigram Graphs)

```
from __future__ import annotations

import math
import re
from collections import Counter
from pathlib import Path
from typing import Any

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import networkx as nx
import numpy as np
from wordcloud import WordCloud

# Color palette tokens matching the UI theme
DARK_BG = "#0E0F12"
SURFACE_BG = "#16181D"
CARD_BG = "#1E2128"
BORDER_COLOR = "#2A2E37"
TEXT_PRIMARY = "#ECE8E1"
TEXT_MUTED = "#9AA0AB"
ACCENT_AMBER = "#FFB020"
SPEAKER_PALETTE = ["#4EA8DE", "#F2A65A", "#7BD389", "#C792EA", "#F07178"]

STOPWORDS = {
    "the", "a", "an", "and", "or", "but", "in", "on", "at", "to", "for", "with", "by", "about",
    "against", "between", "into", "through", "during", "before", "after", "above", "below", "from",
    "up", "down", "in", "out", "off", "over", "under", "again", "further", "then", "once", "here",
    "there", "when", "where", "why", "how", "all", "any", "both", "each", "few", "more", "most",
    "other", "some", "such", "no", "nor", "not", "only", "own", "same", "so", "than", "too", "very",
    "s", "t", "can", "will", "just", "don", "should", "now", "we", "i", "you", "they", "he", "she",
    "it", "our", "my", "your", "their", "his", "her", "its", "us", "me", "him", "them", "re", "ve",
    "ll", "d", "m", "let", "s", "good", "morning", "team", "wrap", "up", "yeah", "okay", "um", "uh"
}


def _clean_tokens(text: str) -> list[str]:
    clean = re.sub(r"\[\d{2}:\d{2}(?::\d{2})?\]|[A-Za-z0-9_-]+:", " ", text)
    tokens = [w.lower() for w in re.findall(r"\b[a-zA-Z]{3,}\b", clean)]
    return [t for t in tokens if t not in STOPWORDS]


def make_wordcloud(transcript: str, width: int = 800, height: int = 400) -> plt.Figure:
    """Generate a word cloud styled to match the dark studio console theme."""
    tokens = _clean_tokens(transcript)
    text = " ".join(tokens) if tokens else "meeting notes discussion"

    def amber_speaker_color_func(word, font_size, position, orientation, random_state=None, **kwargs):
        colors = [ACCENT_AMBER] + SPEAKER_PALETTE
        idx = (hash(word) % len(colors))
        return colors[idx]

    wc = WordCloud(
        width=width,
```

```
        height=height,
        background_color=DARK_BG,
        color_func=amber_speaker_color_func,
        max_words=60,
        prefer_horizontal=0.85,
        random_state=42,
    ).generate(text)

    fig, ax = plt.subplots(figsize=(8, 4), facecolor=DARK_BG)
    ax.imshow(wc, interpolation="bilinear")
    ax.axis("off")
    fig.patch.set_facecolor(DARK_BG)
    plt.tight_layout(pad=0)
    return fig


def make_bigram_graph(transcript: str, top_n: int = 14) -> plt.Figure:
    """Generate a bigram network graph with dark editorial styling."""
    tokens = _clean_tokens(transcript)
    bigrams = list(zip(tokens[:-1], tokens[1:]))
    bigram_counts = Counter(bigrams)

    top_bigrams = [item for item in bigram_counts.most_common(top_n) if item[0][0] != item[0][1]]

    fig, ax = plt.subplots(figsize=(8, 4.5), facecolor=DARK_BG)
    fig.patch.set_facecolor(DARK_BG)
    ax.set_facecolor(DARK_BG)

    if not top_bigrams:
        ax.text(0.5, 0.5, "Insufficient bigrams detected", color=TEXT_MUTED, ha="center", va="center")
        ax.axis("off")
        return fig

    G = nx.Graph()
    for (w1, w2), count in top_bigrams:
        G.add_edge(w1, w2, weight=count)

    pos = nx.spring_layout(G, k=1.3, seed=42)

    # Edge weights
    weights = [G[u][v]["weight"] for u, v in G.edges()]
    max_w = max(weights) if weights else 1
    edge_widths = [1.2 + 2.5 * (w / max_w) for w in weights]

    nx.draw_networkx_edges(
        G, pos, ax=ax,
        edge_color=BORDER_COLOR,
        width=edge_widths,
        alpha=0.85,
    )

    # Nodes
    degrees = dict(G.degree())
    node_sizes = [400 + 150 * deg for deg in degrees.values()]
    node_colors = [ACCENT_AMBER if deg >= 3 else CARD_BG for deg in degrees.values()]
    node_edge_colors = [ACCENT_AMBER if deg >= 3 else SPEAKER_PALETTE[i % len(SPEAKER_PALETTE)] for i, deg in
enumerate(degrees.values())]

    nx.draw_networkx_nodes(
        G, pos, ax=ax,
        node_size=node_sizes,
        node_color=node_colors,
        edgecolors=node_edge_colors,
        linewidths=1.5,
    )


# ... [Truncated: 289 additional lines in production codebase] ...
```

voxintel/audio.py (faster-whisper base int8 ASR with VAD)

```
from __future__ import annotations

import contextlib
import json
import shutil
import subprocess
import tempfile
from functools import lru_cache
from pathlib import Path

SUPPORTED_AUDIO_EXTENSIONS = {".wav", ".mp3"}
SUPPORTED_TEXT_EXTENSIONS = {".txt"}
VOSK_MODEL_DIR = Path(__file__).resolve().parents[1] / "data" / "models" / "vosk-model-small-en-us-0.15"


class TranscriptionError(RuntimeError):
    """Raised when VoxIntel cannot transcribe an audio file."""


def validate_input_file(file_path: str | Path) -> Path:
    path = Path(file_path)
    if not path.exists():
        raise FileNotFoundError(f"File not found: {path}")
    if path.suffix.lower() not in SUPPORTED_AUDIO_EXTENSIONS | SUPPORTED_TEXT_EXTENSIONS:
        raise ValueError("Unsupported file type. Use .wav, .mp3, or .txt.")
    return path


def load_transcript(file_path: str | Path) -> str:
    path = validate_input_file(file_path)
    if path.suffix.lower() != ".txt":
        raise ValueError("Transcript loading expects a .txt file.")
    return path.read_text(encoding="utf-8").strip()


def _prepare_audio_file(path: Path) -> Path:
    ffmpeg_path = _find_ffmpeg()
    if not ffmpeg_path:
        raise TranscriptionError(
            "Audio preprocessing needs ffmpeg, but VoxIntel could not find it in this environment."
        )

    temp_dir = Path(tempfile.mkdtemp(prefix="voxintel_audio_"))
    temp_path = temp_dir / "decoded.wav"

    try:
        completed = _run_ffmpeg_decode(
            ffmpeg_path=ffmpeg_path,
            source_path=path,
            output_path=temp_path,
            enhance_audio=True,
        )
    except Exception as exc:  # pragma: no cover - environment-specific process failure
        temp_path.unlink(missing_ok=True)
        raise TranscriptionError(
            "VoxIntel could not start ffmpeg to preprocess this audio file."
        ) from exc

    if completed.returncode != 0 or not temp_path.exists() or temp_path.stat().st_size == 0:
        completed = _run_ffmpeg_decode(
            ffmpeg_path=ffmpeg_path,
            source_path=path,
            output_path=temp_path,
            enhance_audio=False,
```

```
        )

    if completed.returncode != 0 or not temp_path.exists() or temp_path.stat().st_size == 0:
        temp_path.unlink(missing_ok=True)
        temp_dir.rmdir()
        stderr_tail = (completed.stderr or "").strip().splitlines()[-1:] if completed.stderr else []
        detail = f" Details: {stderr_tail[0]}" if stderr_tail else ""
        raise TranscriptionError(
            "VoxIntel could not preprocess this audio file with ffmpeg." + detail
        )

    return temp_path


def _run_ffmpeg_decode(ffmpeg_path: Path, source_path: Path, output_path: Path, enhance_audio: bool) ->
subprocess.CompletedProcess:
    command = [
        str(ffmpeg_path),
        "-y",
        "-i",
        str(source_path),
    ]
    if enhance_audio:
        command.extend(
            [
                "-af",
                "highpass=f=80,lowpass=f=3800,afftdn=nf=-20,dynaudnorm",
            ]
        )
    command.extend(
        [
            "-acodec",
            "pcm_s16le",
            "-ar",
            "16000",
            "-ac",
            "1",
            str(output_path),
        ]
    )
    return subprocess.run(
        command,
        capture_output=True,
        text=True,
        check=False,
    )


def _find_ffmpeg() -> Path | None:
    system_ffmpeg = shutil.which("ffmpeg")
    if system_ffmpeg:
        return Path(system_ffmpeg)

    candidate_paths = [
        Path(

r"C:\Users\eshan\AppData\Local\Microsoft\WinGet\Packages\Gyan.FFmpeg_Microsoft.Winget.Source_8wekyb3d8bbwe\ffmpeg
-8.1-full_build\bin\ffmpeg.exe"
        ),

# ... [Truncated: 332 additional lines in production codebase] ...
```

voxintel/insights.py (TextRank Summarization & Action Item Extraction)

```
from __future__ import annotations
```

```
import re
from collections import Counter

import networkx as nx

from .ml import AISummaryUnavailable, generate_ai_summary
from .nlp import clean_sentences, clean_transcript, split_sentences, tokenize

ACTION_PATTERNS = [
    re.compile(
        r"(?i)^(?P<person>[A-Z][a-z]+(?:\s[A-Z][a-z]+)?)\s+(?:will|should|needs to|must|has to|is going
to)\s+(?P<task>.+?)(?:\s+(?:by|before|on)\s+(?P<deadline>[^.]+?))?[.]?$"
    ),
    re.compile(
        r"(?i)^(?:(?:once|after|as soon
as|when|regarding|before|if)\s+[^,;]+,\s*)?(?:i'll|i\s+will|we'll|we\s+will|i\s+can|we\s+can|i\s+should|i\s+need\
s+to|we\s+need\s+to|i\s+am\s+going\s+to|we\s+are\s+going\s+to)\s+(?P<task>.+?)(?:\s+(?:by|before|on|as soon
as)\s+(?P<deadline>[^.]+?))?[.]?$"
    ),
    re.compile(
        r"(?i)^action item[:\-]?\s*(?P<person>[A-Z][a-z]+)?\s*(?:will|should|needs
to)?\s*(?P<task>.+?)(?:\s+(?:by|before|on)\s+(?P<deadline>[^.]+?))?[.]?$"
    ),
]

DECISION_PATTERN = re.compile(r"(?i)\b(decide|decided|decision|agreed|approved|finalized|confirmed)\b")
DECISION_STRONG_PATTERN = re.compile(
    r"(?i)\b(decide|decided|decision|approved|agreed|finalized|confirmed|moving forward|temporary
workaround|workaround)\b"
)
DATE_PATTERN = re.compile(
    r"\b(?:\d{1,2}[/-]\d{1,2}[/-]\d{2,4}|"
    r"(?:jan|feb|mar|apr|may|jun|jul|aug|sep|oct|nov|dec)[a-z]*\s+\d{1,2}(?:,\s*\d{4})?)\b",
    re.IGNORECASE,
)
SPEAKER_PATTERN = re.compile(r"(?m)^\s*([A-Z][a-z]+(?:\s[A-Z][a-z]+)?)\s*:")
SPEAKER_INLINE_PATTERN = re.compile(r"\b([A-Z][a-z]+(?:\s[A-Z][a-z]+)?)\s*:")
FULL_NAME_PATTERN = re.compile(r"\b([A-Z][a-z]+ [A-Z][a-z]+)\b")
TOPIC_CUE_PATTERN = re.compile(

r"(?i)\b(issue|issues|bug|bugs|authentication|ui|design|release|timeline|deadline|demo|support|testing|review|fix
|plan|notes|documentation)\b"
)
DEADLINE_PATTERN = re.compile(
    r"(?i)\b(by tomorrow|tomorrow|next week|next month|this week|today|by [A-Z][a-
z]+day(?:\s+at\s+\d{1,2}(?::\d{2})?\s*(?:AM|PM))?|\d{1,2}(?::\d{2})?\s*(?:AM|PM)(?:\s+today)?)\b"
)
WEAK_ACTION_PATTERN = re.compile(r"(?i)\b(we'?ll try|we'?ll ensure|sounds good|good idea|good point|maybe|should
discuss|let'?s discuss)\b")
VAGUE_DECISION_PATTERN = re.compile(r"(?i)\b(maybe|might|should|let'?s discuss|perhaps|could)\b")
COMMON_FIRST_NAMES = {
    "alex","amy","andrew","anna","asha","ben","chris","christy","daniel","david","dove","eric","fabian",
    "gabe","george","jessica","john","josh","karina","kenny","meera","paul","rahul","scott","sid","virginia"
}


def infer_speaker_activity(transcript: str) -> list[dict[str, int]]:
    speaker_counts = Counter(SPEAKER_PATTERN.findall(transcript))
    return [{"speaker": speaker, "mentions": count} for speaker, count in speaker_counts.most_common()]


def normalize_for_summary(text: str) -> str:
    normalized = re.sub(r"\s+", " ", text).strip()
    normalized = SPEAKER_INLINE_PATTERN.sub("", normalized)
    normalized = re.sub(r"\b(today|hello everyone|good morning team)\b[:,]?", "", normalized,
flags=re.IGNORECASE)
    normalized = re.sub(r"\s+", " ", normalized).strip(" ,")
    return normalized
```

```
def _sentence_similarity(sentence_a: str, sentence_b: str) -> float:
    tokens_a = set(tokenize(sentence_a))
    tokens_b = set(tokenize(sentence_b))
    if not tokens_a or not tokens_b:
        return 0.0
    overlap = len(tokens_a & tokens_b)
    return overlap / max((len(tokens_a) * len(tokens_b)) ** 0.5, 1.0)


def _textrank_scores(sentences: list[str]) -> dict[int, float]:
    if len(sentences) <= 1:
        return {0: 1.0} if sentences else {}

    graph = nx.Graph()
    for index in range(len(sentences)):
        graph.add_node(index)
    for left in range(len(sentences)):
        for right in range(left + 1, len(sentences)):
            similarity = _sentence_similarity(sentences[left], sentences[right])
            if similarity > 0:
                graph.add_edge(left, right, weight=similarity)

    if graph.number_of_edges() == 0:
        return {index: 1.0 for index in range(len(sentences))}
    return nx.pagerank(graph, weight="weight")


def _generate_rule_based_summary(sentences: list[str], tokens: list[str]) -> str:
    if not sentences:
        return "No summary available because the transcript did not contain enough content."

    keyword_weights = Counter(tokens)
    textrank_scores = _textrank_scores(sentences)
    scored_sentences = _rank_sentences(_build_sentence_blocks(sentences), keyword_weights, textrank_scores,
is_block=True)
    selected = _select_diverse_sentences(scored_sentences, max_sentences=min(4, len(scored_sentences)))
    ordered = [sentence for _, _, sentence, _ in sorted(selected, key=lambda item: item[1])]
    return _final_summary_pass(". ".join(sentence.strip(" .") for sentence in ordered if sentence.strip()))


def _build_sentence_blocks(sentences: list[str], block_size: int = 3) -> list[str]:
    if len(sentences) <= block_size:
        return sentences
    blocks: list[str] = []
    for index in range(0, len(sentences), block_size):
        block = " ".join(sentences[index : index + block_size]).strip()
        if block:
            blocks.append(block)
    return blocks


def _final_summary_pass(text: str) -> str:
    text = re.sub(r"\s+", " ", text).strip()
    text = re.sub(r"\.\s+([a-z])", lambda m: ". " + m.group(1).upper(), text)
    text = re.sub(r"([A-Za-z])\s+([A-Z][a-z]+ed)\b", r"\1. \2", text)
    parts = split_sentences(text)
    deduped: list[str] = []
    seen = set()
    for part in parts:
        normalized = re.sub(r"\s+", " ", part).strip(" .")

# ... [Truncated: 339 additional lines in production codebase] ...
```

voxintel/ollama_polish.py (Local Ollama Third-Person MoM Synthesis)

```
from __future__ import annotations

import json
import os
import re
from pathlib import Path
from urllib import request, error

from .exports import render_mom_html, render_mom_pdf
from .optimized_pipeline import OptimizedPipelineError, run_optimized_voxintel

DEFAULT_OLLAMA_HOST = "http://127.0.0.1:11434"
DEFAULT_OLLAMA_MODEL = "llama3.2:3b"


class OllamaPolishError(RuntimeError):
    """Raised when local Ollama polishing cannot continue."""


def _get_ollama_settings() -> tuple[str, str]:
    host = os.getenv("OLLAMA_HOST", DEFAULT_OLLAMA_HOST).rstrip("/")
    env_model = os.getenv("OLLAMA_MODEL", "").strip()
    if env_model:
        return host, env_model

    try:
        req = request.Request(f"{host}/api/tags", headers={"Content-Type": "application/json"}, method="GET")
        with request.urlopen(req, timeout=1.5) as resp:
            data = json.loads(resp.read().decode("utf-8"))
            installed = [m.get("name", "") for m in data.get("models", [])]
            for candidate in ["llama3.2:3b", "llama3.2", "gemma3:1b", "gemma3", "gemma2", "qwen2.5:0.5b",
"qwen2.5", "phi3", "phi:latest", "mistral"]:
                for inst in installed:
                    if candidate in inst:
                        return host, inst
            if installed:
                return host, installed[0]
    except Exception:
        pass

    return host, DEFAULT_OLLAMA_MODEL


def resolve_speaker_mapping(segments: list[dict], attendees: list[str] | None = None) -> dict[str, str]:
    """Map anonymous Speaker clusters (Speaker 1, 2...) to real attendee names using conversational
evidence."""
    if not attendees:
        return {}

    mapping: dict[str, str] = {}
    assigned_attendees: set[str] = set()
    clusters = sorted(list(set(s.get("speaker", "") for s in segments if s.get("speaker",
"").startswith("Speaker"))))

    # 1. Direct self-introductions ("This is [Name]", "I am [Name]")
    for att in attendees:
        att_lower = att.lower()
        for s in segments:
            text_lower = (s.get("text") or "").lower()
            spk = s.get("speaker")
            if not spk or spk in mapping:
                continue
            if f"this is {att_lower}" in text_lower or f"i am {att_lower}" in text_lower or f"i'm {att_lower}"
in text_lower:
                mapping[spk] = att
                assigned_attendees.add(att)
                break
```

```
    # 2. Conversational Adjacency Pairs:
    # A) "Thanks, [Name]" / "Thank you, [Name]" -> the PRECEDING turn belonged to [Name]
    for i, s in enumerate(segments):
        if i == 0:
            continue
        txt = (s.get("text") or "").lower()
        prev_spk = segments[i - 1].get("speaker")
        if not prev_spk or prev_spk in mapping:
            continue
        for att in attendees:
            if att in assigned_attendees:
                continue
            if re.search(rf"\b(?:thanks|thank you),?\s+{att.lower()}\b", txt):
                mapping[prev_spk] = att
                assigned_attendees.add(att)
                break

    # B) "[Name], what/how/can/could/is/are/do/did...?" -> the NEXT turn answering belongs to [Name]
    for i, s in enumerate(segments[:-1]):
        txt = (s.get("text") or "").lower()
        next_s = segments[i + 1]
        next_spk = next_s.get("speaker")
        if not next_spk or next_spk in mapping:
            continue
        next_txt = (next_s.get("text") or "").lower()
        for att in attendees:
            if att in assigned_attendees:
                continue
            if re.search(rf"\b{att.lower()},\s*(?:what|how|can|could|is|are|do|did|would|will)\b", txt):
                if any(next_txt.startswith(p) for p in ["i ", "i'll", "i will", "we ", "we'll", "from ", "the
", "over ", "yes", "sure", "thanks"]):
                    mapping[next_spk] = att
                    assigned_attendees.add(att)
                    break

    # 3. Cross-referencing conversational commitments:
    # E.g. Speaker A says: "Ravi handles support notifications, George patches login"
    # -> Speaker A is not Ravi and not George (it's the meeting chair/other attendee).
    # And the turn committing to support notifications belongs to Ravi!
    for s_third in segments:
        txt_third = (s_third.get("text") or "").lower()
        spk_third = s_third.get("speaker")

        # Check if speaker mentions multiple other attendees (chairperson pattern)
        mentions = [a for a in attendees if a.lower() in txt_third]
        if len(mentions) >= 2 and spk_third and spk_third not in mapping:
            chair_candidates = [a for a in attendees if a not in mentions and a not in assigned_attendees]
            if len(chair_candidates) == 1:
                mapping[spk_third] = chair_candidates[0]
                assigned_attendees.add(chair_candidates[0])

        for att in attendees:
            if att in assigned_attendees:
                continue
            att_lower = att.lower()
            m = re.findall(rf"\b{att_lower}\s+(?:will|handles|patches|is going
to|takes|releases|completes)\s+([^.,;]+)", txt_third)
            for mention_context in m:

# ... [Truncated: 547 additional lines in production codebase] ...
```